

Deterministic Parallel Transaction Execution for Logos Execution Zone: A Comparative and Experimental Study

Daniel Garbanzo

Instituto Tecnológico de Costa Rica
Master's Program in Computer Science – Parallel Computing (MC-8836)

Abstract. Modern blockchain execution layers must increase transaction throughput without changing deterministic state-machine semantics. A central challenge is that transactions in an ordered block may access shared state such as accounts, nonces, storage keys, nullifiers, commitments, and program metadata. This paper presents CAPE, a Conflict-Aware Parallel Executor model for studying deterministic parallel transaction execution using Logos Execution Zone as the main architectural case study. CAPE defines a common transaction and conflict model and compares three strategies: sequential execution, static conflict-aware batching, and optimistic execution with ordered commit and re-execution. The Rust prototype generates controlled workloads with disjoint access, hotspot contention, Zipfian popularity, token transfers, and simplified private-state operations. Across 990 benchmark rows, including 360 local rows on an Apple M2 Pro and 630 Kabré rows on a Dribe Xeon node with 36 allocated CPUs, all parallel executions matched the sequential final state hash and per-transaction outcomes. The optimistic executor achieved the best measured local throughput, 731,729.63 transactions per second, on a no-conflict workload with 8 threads and 5,000 transactions per block. The Kabré run reached 583,276.55 transactions per second on the same workload family with 16 threads, validating correctness and high-thread stress behavior but not exceeding the local peak. Static scheduling was useful only when batches remained large, but barrier-like workloads collapsed into small batches. The main contributions are a comparative taxonomy of blockchain parallel execution techniques, a Logos-oriented conflict model, a reproducible Rust benchmark, and a practical adaptation path for LEZ that starts from safe parallel validation and ordered state-diff commit.

Keywords: Parallel computing · Blockchain execution · Determinism · Scheduling · Optimistic concurrency control · Logos Execution Zone

1 Introduction

Blockchain nodes implement replicated state machines. Given the same initial state and the same ordered block, honest nodes must compute the same final state. The simplest execution policy is sequential: validate and apply each

transaction in canonical block order. This policy is easy to reason about and deterministic by construction, but it underuses multicore hardware and can limit throughput.

Modern blockchain systems expose several approaches to this problem. Solana Sealevel schedules transactions using declared account access metadata [16]. Aptos Block-STM adapts software transactional memory to ordered blockchain blocks [7]. Cosmos SDK documents similar Block-STM integration concerns around serial equivalence and application-specific validation [5]. Monad preserves Ethereum-style ordered semantics while using optimistic execution and asynchronous execution [11, 10]. Sei describes an optimistic concurrency engine with dependency estimation and conflict resolution [14]. Sui exposes independence through an object ownership model [17].

This paper studies these ideas from a parallel computing perspective and applies them to Logos Execution Zone (LEZ), a Rust-based programmable execution environment with public and private state [9]. LEZ is an appropriate case study because its execution model separates validation from state mutation: validation can produce state diffs that are later committed to public accounts, nonces, commitments, nullifiers, and program state. This structure suggests a two-phase design: perform safe validation or computation in parallel, then commit deterministic diffs in canonical order.

The project does not attempt to implement a production Logos node or compare directly against production systems whose internals are unavailable. Instead, it contributes a controlled experimental framework for reasoning about deterministic parallel execution. The paper combines a taxonomy of existing blockchain execution designs, a Logos-oriented conflict model, a reproducible Rust prototype, and a dual-environment evaluation. This scope is intentionally narrower than a production deployment, but it is sufficient to answer the central parallel-computing question: when does deterministic parallel execution preserve correctness while improving throughput, and when do conflicts and ordered commit dominate?

2 Background and Related Work

Parallel computing studies how work can be decomposed while managing synchronization, communication, load balance, and memory contention [12]. Speedup is measured as T_1/T_p , and parallel efficiency is speedup divided by the number of workers. Amdahl-style limits appear when some fraction of work must remain serial. In CAPE, serial work includes ordered commit, validation of read versions, and unavoidable conflicts.

Bulk Synchronous Parallel (BSP) provides useful vocabulary for static batching: independent work is executed in a superstep, followed by synchronization and ordered commit [18]. LogP emphasizes that communication and synchronization overhead are not free [6]. Roofline modeling is not the evaluation method here, but it motivates distinguishing compute-heavy transactions from metadata-

bound schedulers [19]. To avoid measuring only framework overhead, the prototype supports configurable artificial compute cost.

Optimistic concurrency control (OCC) assumes conflicts are uncommon: transactions execute speculatively and validate before commit [8]. Software transactional memory applies related ideas to shared-memory programs by tracking read and write sets [15]. Block-STM applies this approach to blockchain blocks with predetermined order, validating speculative reads and re-executing invalidated transactions [7]. Earlier smart-contract concurrency work also explored optimistic and object-based transactional execution [2, 1, 13].

Table 1 summarizes the design points relevant to CAPE.

Table 1. Taxonomy of deterministic parallel blockchain execution strategies.

System	Dependency information	Strategy	Determinism policy
Sequential	none	ordered execution	canonical by construction
Solana	declared accounts	static scheduling	account access metadata
Aptos	dynamic read/write sets	Block-STM	preset block order
Cosmos SDK	store tracking	Block-STM option	AppHash equivalence
Monad	tracked inputs	optimistic execution	Ethereum-equivalent order
Sei	estimated/runtime sets	OCC hybrid	deterministic resolution
Sui	object ownership	object-level independence	ownership constraints
Logos LEZ	accounts, diffs, nullifiers	case-study target	ordered state transition

3 Problem Description

Let a block be

$$B = [tx_1, tx_2, \dots, tx_n]$$

and let S_0 be the initial state. Canonical sequential execution defines

$$S_i = \text{apply}(\text{validate}(tx_i, S_{i-1}))$$

for each transaction tx_i . A parallel executor is correct only if

$$\text{parallel}(B, S_0).state = \text{sequential}(B, S_0).state$$

and

$$\text{parallel}(B, S_0).outcomes = \text{sequential}(B, S_0).outcomes.$$

The difficulty is that blockchain transactions access shared state. In a Logos-inspired model, conflicts may involve public account balances, signer nonces, storage keys, deployed program code, private nullifiers, commitment roots, commitment slots, system clocks, and global configuration. Some work is naturally parallel, such as stateless checks, signature verification, proof verification, and computation over snapshots. Other work must respect canonical order because it changes the replicated state.

This leads to four research questions:

1. How much speedup can static conflict-aware scheduling obtain under different contention patterns?
2. How does optimistic execution behave as conflict rate increases?
3. Which LEZ-inspired state components can be parallelized safely and which require ordered commit?
4. What overheads dominate when available parallelism is low?

4 CAPE: Conflict-Aware Parallel Executor

CAPE is a controlled experimental model, not a production blockchain runtime. Its state model includes accounts, nonces, nullifiers, commitments, program code, program storage, a system clock, global configuration, and a per-key version map. Each conflict-relevant item is represented as a `StateKey`. Every write increments the version of the written key. A canonical state hash is computed over sorted state components, avoiding dependence on hash-map iteration order.

Transactions carry a kind, signer, optional receiver, declared reads, declared writes, consumed nullifiers, produced commitments, optional program identifiers, amount, and compute cost. The prototype implements five transaction families: public transfers, private transfers, program calls, program deployments, and system transactions. Rejected transactions produce no state diff but still produce deterministic outcomes.

4.1 Conflict Relation

Two transactions conflict when their effects cannot be reordered without potentially changing sequential semantics. CAPE uses the following rule:

$$\text{conflict}(tx_i, tx_j) \Leftrightarrow (W_i \cap W_j \neq \emptyset) \vee (W_i \cap R_j \neq \emptyset) \vee (R_i \cap W_j \neq \emptyset)$$

with additional treatment for duplicate nullifiers, program deployment, system clock, and global configuration. System and deployment transactions are barriers.

4.2 Execution Strategies

Sequential baseline. The sequential executor clones the initial state, executes each transaction in block order, applies its diff, records the outcome, and computes the final hash. It is the correctness oracle.

Static conflict-aware scheduling. The static executor greedily builds batches from declared read/write sets. A transaction is added to the current batch only if it has no read/write or write/write overlap with the accumulated batch sets. Barrier transactions form single-transaction batches. Each batch executes in parallel over a snapshot and commits diffs in canonical order.

Optimistic ordered commit. The optimistic executor speculatively executes the whole block in parallel over an initial snapshot. Each transaction records the versions of keys it read. During ordered commit, a transaction commits its speculative diff only if all read versions still match the current state. Otherwise, it is re-executed against the current state and then committed. This preserves the same observable result as sequential execution while allowing useful speculative work.

4.3 Complexity

Sequential execution is $O(nC_{tx})$, where C_{tx} is transaction execution cost. Naive static conflict graph construction is $O(n^2)$, but the prototype uses accumulated read/write sets per batch to avoid pairwise graph construction in the common path. Optimistic execution performs one speculative pass and may re-execute invalid transactions, so its cost is approximately $O(nC_{tx} + rC_{tx} + V)$, where r is the number of re-executions and V is read-version validation overhead.

5 Experimental Methodology

The prototype was implemented in Rust and evaluated first on macOS Darwin 25.5.0 with an Apple M2 Pro, 10 logical threads, 16 GiB RAM, and Rust 1.96.0. This local environment represents commodity personal hardware, which is closer to a decentralization-friendly operator or validator setting. The benchmark matrix used five workloads:

- **No-conflict:** disjoint sender/receiver accounts.
- **Hotspot:** many transactions write the same sender account.
- **Zipfian:** account popularity follows a skewed distribution.
- **Token-transfer:** random sender/receiver token transfers.
- **Private-state:** simplified nullifier consumption and commitment append operations.

The local matrix ran 3 executors, 5 workloads, 2 block sizes (1,000 and 5,000), 4 thread counts (1, 2, 4, 8), and 3 deterministic seeds, for 360 rows. Each run recorded elapsed time, throughput, scheduling time, execution time, validation time, commit time, conflicts detected by the static scheduler, optimistic re-executions, accepted/rejected transactions, final state hash, and correctness against the sequential baseline.

A second run was executed on Kabré, CeNAT’s computational cluster, whose public specification lists 620 cores, 1,004 threads, 224 TB of storage, and a 10 Gbps interconnection network [3]. Kabré documentation directs heavy tasks to Slurm queues rather than login nodes and lists Dribe as a Xeon partition [4]. The Kabré experiment used one Dribe node through Slurm with 36 allocated CPUs, Rust 1.96.0 from a user-level rustup installation, the same workloads and block sizes, thread counts 1, 2, 4, 8, 16, 32, and 36, and 3 deterministic seeds, for 630 rows. This second environment should be interpreted as institutional scalability stress testing, not as a requirement for decentralized node operation.

Artifact availability. The complete source code, benchmark scripts, raw CSV outputs, generated figures, Kabré Slurm script, and reproducibility notes are available at <https://github.com/bitfalt/cape-lez-parallel-executor>. The repository is intended to make the methodology and reported measurements inspectable without changing the claims made in this paper.

6 Results

All 990 benchmark rows reported correct final state hashes and transaction outcomes. Therefore, the parallel strategies preserved canonical sequential semantics for all tested workloads in both environments.

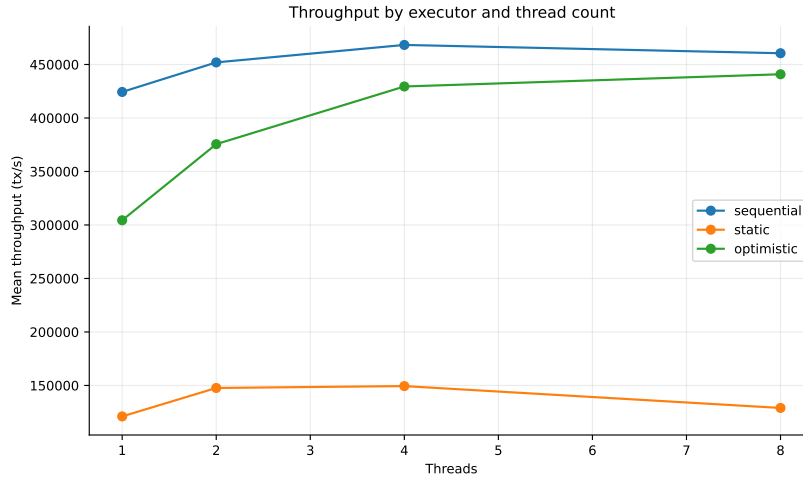


Fig. 1. Mean throughput by executor and thread count.

Figure 1 shows that the optimistic executor benefited most from additional threads. Its mean throughput increased from 304,392 tx/s at 1 thread to 440,860 tx/s at 8 threads. The best individual run reached 731,729.63 tx/s for the no-conflict workload with 5,000 transactions and 8 threads. The sequential baseline remained comparatively stable because its implementation ignores the thread count and is included as a reference.

The speedup results in Figure 2 show a clear distinction between static and optimistic execution. At 8 threads, optimistic execution had a mean speedup of 1.113 across workload/block-size combinations. Static scheduling had a mean speedup of 0.366 because high-contention and barrier-like workloads produced many small batches and paid scheduling overhead without enough parallel work.

Figure 3 explains the optimistic executor’s behavior. No-conflict workloads required zero re-executions. Hotspot and private-state workloads required nearly

Table 2. Mean throughput by executor and thread count, aggregated across workloads and block sizes.

Executor	1 thread	2 threads	4 threads	8 threads
Sequential	424,334	451,956	468,282	460,522
Static	121,061	147,654	149,415	128,982
Optimistic	304,392	375,526	429,441	440,860

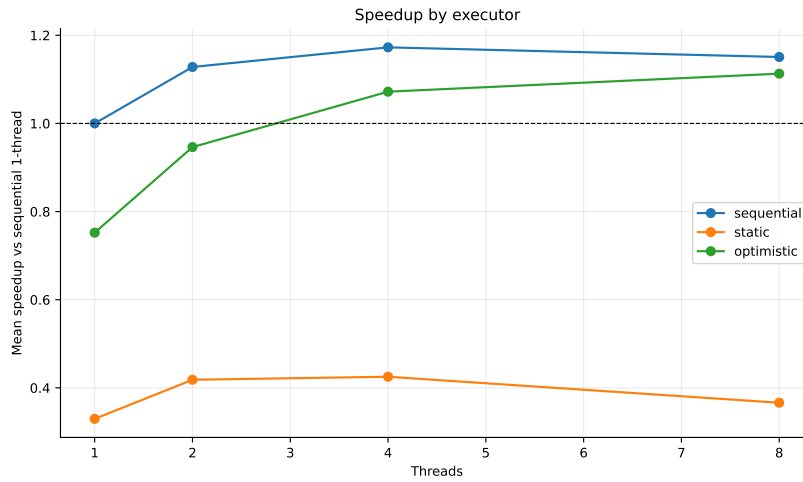


Fig. 2. Speedup relative to the sequential 1-thread baseline.

Table 3. 8-thread speedup relative to sequential 1-thread execution.

Workload	Block size	Static	Optimistic
Hotspot	1,000	0.04	0.72
Hotspot	5,000	0.03	0.82
No-conflict	1,000	1.34	1.59
No-conflict	5,000	0.69	1.51
Private-state	1,000	0.05	1.08
Private-state	5,000	0.02	0.87
Token-transfer	1,000	0.72	1.47
Token-transfer	5,000	0.59	1.49
Zipfian	1,000	0.11	0.75
Zipfian	5,000	0.07	0.82

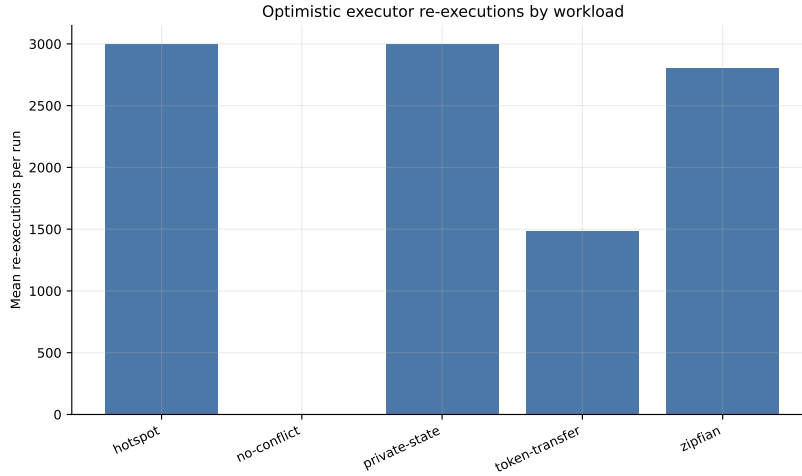


Fig. 3. Mean optimistic re-executions by workload.

every transaction after the first to re-execute because all transactions touched a shared key: the hotspot account or the commitment root/nullifier-related state. Token-transfer workloads were mixed, with an average of 1,487.83 re-executions across runs, while Zipfian workloads averaged 2,806.33 re-executions because popular accounts produced repeated invalidation.

Figure 4 shows that static scheduling spent a mean of 2.193 ms in scheduling and 1.352 ms in commit, while optimistic execution spent 0.284 ms in validation and 0.708 ms in commit. Execution time dominated both strategies, but scheduling overhead became visible when batches were small.

The results support the central hypothesis: deterministic parallel execution helps when enough independent work exists, but contention, validation, re-execution, and ordered commit can remove the benefit. For LEZ-like systems, the safest near-term adaptation is to parallelize stateless validation and computation while preserving ordered state-diff commit.

6.1 Dual-Environment Kabré Validation

Table 4. Local and Kabré benchmark summary.

Environment	Rows	Threads	Failures	Best tx/s
Apple M2 Pro local	360	1–8	0	731,729.63
Kabré Dribe Xeon	630	1–36	0	583,276.55

Table 4 and Figure 6 show that Kabré is valuable as an additional correctness and scalability environment, but it did not improve the peak throughput of this shared-memory prototype. The best local and Kabré runs were both optimistic no-conflict executions with block size 5,000; the local peak occurred

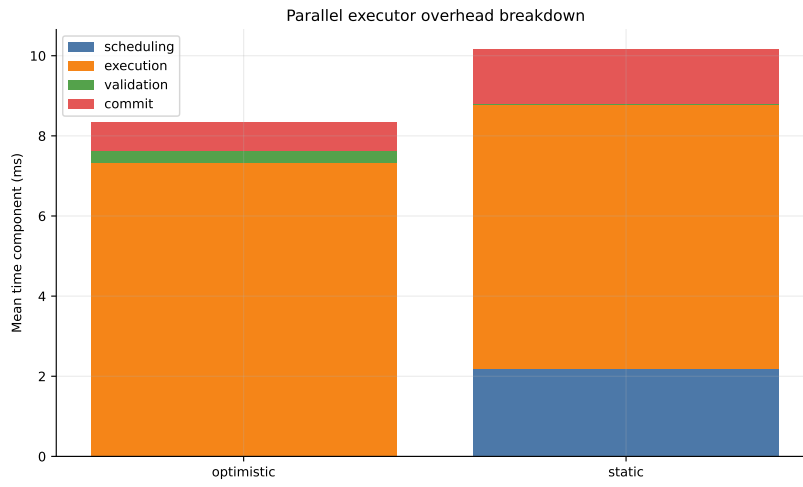


Fig. 4. Mean timing component breakdown for parallel executors.

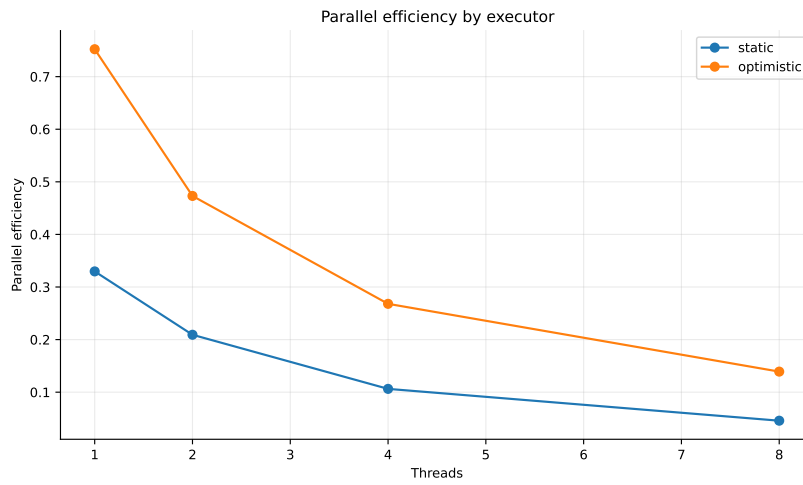


Fig. 5. Parallel efficiency for static and optimistic executors.

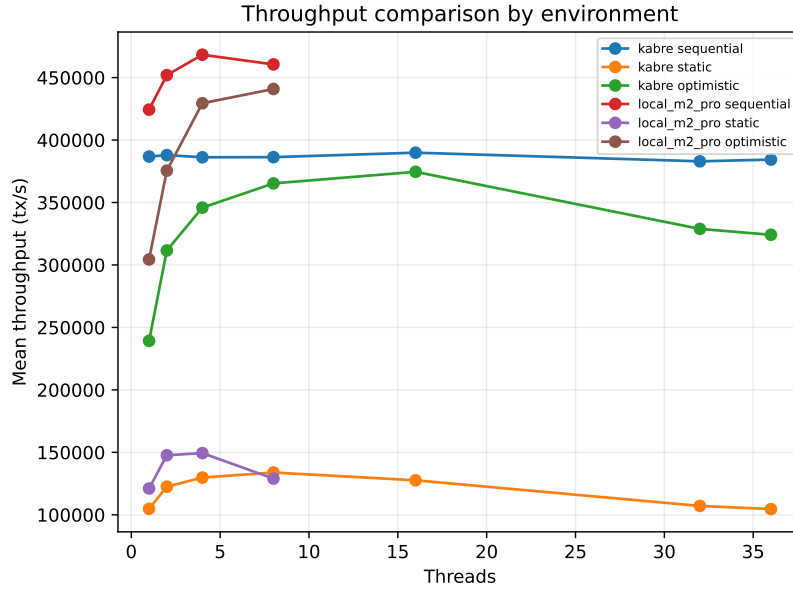


Fig. 6. Mean throughput comparison between the local Apple M2 Pro run and the Kabré Dribe run.

at 8 threads, while the Kabré peak occurred at 16 threads. At common optimistic thread counts, Kabré reached 78.6% to 83.0% of the local throughput. Within the Kabré-only high-thread range, optimistic mean throughput increased from 239,213 tx/s at 1 thread to 374,558 tx/s at 16 threads, then declined to 328,869 tx/s at 32 threads and 324,158 tx/s at 36 threads. This suggests that the prototype’s overheads, memory behavior, and ordered commit limits dominate after moderate parallelism. The result is still useful: the local run supports the decentralization-oriented claim, while the Kabré run shows that the same semantics remain correct under a larger institutional CPU allocation.

7 Contributions

The main contribution of this project is an experimental framework for studying deterministic parallel transaction execution in a way that is specific enough to be useful for Logos Execution Zone, but general enough to compare against established blockchain execution strategies. Rather than treating parallel execution as a single technique, the paper separates declared-access scheduling, optimistic execution, object-centric execution, asynchronous execution, and sequential execution into distinct design points. This taxonomy makes the tradeoffs explicit: some systems expose independence before execution, others discover conflicts

dynamically, and all deterministic blockchains must preserve a canonical state transition.

A second contribution is the conflict model used by CAPE. The model covers public account balances, signer nonces, arbitrary state keys, nullifiers, commitments, program code, and global system state. This is important because a Logos-like environment combines ordinary public state with private-state mechanisms, and those components do not have the same parallelization boundaries. The model therefore distinguishes work that can be safely computed over snapshots from state changes that must be committed in canonical order.

The third contribution is the Rust prototype itself. CAPE implements a sequential correctness oracle, a static conflict-aware scheduler, and an optimistic executor with ordered commit and re-execution. The implementation is not presented as a production runtime; its value is that each executor operates over the same transaction and state model, making the comparison controlled and reproducible. The public artifact also includes the benchmark scripts, raw CSV outputs, generated figures, and the Kabré Slurm script used for the second environment.

Finally, the evaluation contributes empirical evidence about where the proposed approach is effective. Across 990 benchmark rows, including both local and Kabré executions, all parallel runs matched the sequential final state hash and transaction outcomes. The results show that optimistic execution is the strongest strategy when enough independent work exists, while static scheduling collapses under hotspot, private-state, or barrier-heavy workloads. The dual-environment comparison adds an important practical nuance: more institutional hardware did not automatically produce a higher peak throughput, which reinforces that contention, memory behavior, and ordered commit are central limits for this class of execution engines.

8 Conclusions and Future Work

CAPE demonstrates that deterministic parallel blockchain execution can preserve sequential semantics while exploiting multicore hardware under favorable dependency patterns. The optimistic executor was the most robust strategy in the experiment, especially on no-conflict and token-transfer workloads. Static scheduling was sensitive to declared conflicts and performed poorly when barriers or shared keys collapsed the schedule into small batches. The Kabré experiment strengthens the correctness result and adds a high-thread stress test, but the local Apple M2 Pro run remained the best peak-throughput environment for this prototype. Therefore, the most accurate conclusion is not that HPC hardware is required, but that deterministic parallel execution is semantically viable and that real performance depends strongly on hardware, contention, and ordered-commit overhead.

For Logos Execution Zone, the practical adaptation path should start with parallel stateless checks, proof or signature verification, transaction classification, and conflict metadata construction. Ordered commit should remain the seman-

tic boundary for public account updates, signer nonces, nullifiers, commitments, program deployments, and global system state. Future work should replace synthetic execution with traces from real LEZ workloads, model proof-verification cost more accurately, evaluate hybrid scheduling, reduce clone overhead with persistent snapshots or versioned storage, and test against adversarial dependency patterns.

References

1. Anjana, P.S., Attiya, H., Kumari, S., Peri, S., Somani, A.: Efficient concurrent execution of smart contracts in blockchains using object-based transactional memory (2019), <https://arxiv.org/abs/1904.00358>
2. Anjana, P.S., Kumari, S., Peri, S., Rathor, S., Somani, A.: An efficient framework for optimistic concurrent execution of smart contracts (2018), <https://arxiv.org/abs/1809.01326>
3. Centro Nacional de Alta Tecnología: Kabre: Cluster cnca/cenat. Cluster information page (2026), <https://kabre.cenat.ac.cr/en>, accessed: 2026-06-20
4. CNCA: Entendiendo el sistema de colas de kabre. Online documentation (2025), <https://kabre-documentation.readthedocs.io/es/latest/queuesSystem.html>, accessed: 2026-06-20
5. Cosmos SDK Documentation: Block-STM: Parallel transaction execution. Online documentation (2026), <https://docs.cosmos.network/sdk/latest/experimental/blockstm>, accessed: 2026-06-14
6. Culler, D., Karp, R., Patterson, D., Sahay, A., Schauser, K.E., Santos, E., Subramonian, R., von Eicken, T.: LogP: Towards a realistic model of parallel computation. In: Proceedings of the Fourth ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming. pp. 1–12 (1993). <https://doi.org/10.1145/155332.155333>
7. Gelashvili, R., Spiegelman, A., Xiang, Z., Danezis, G., Li, Z., Malkhi, D., Xia, Y., Zhou, R.: Block-STM: Scaling blockchain execution by turning ordering curse to a performance blessing (2022), <https://arxiv.org/abs/2203.06871>
8. Kung, H.T., Robinson, J.T.: On optimistic methods for concurrency control. ACM Transactions on Database Systems **6**(2), 213–226 (1981). <https://doi.org/10.1145/319566.319567>
9. Logos Blockchain: Logos execution zone repository and readme. GitHub repository (2026), <https://github.com/logos-blockchain/logos-execution-zone>, accessed: 2026-06-14
10. Monad Documentation: Asynchronous execution. Online documentation (2026), <https://docs.monad.xyz/monad-arch/consensus/asynchronous-execution>, accessed: 2026-06-14
11. Monad Documentation: Parallel execution. Online documentation (2026), <https://docs.monad.xyz/monad-arch/execution/parallel-execution>, accessed: 2026-06-14
12. Pacheco, P.S.: An Introduction to Parallel Programming. Morgan Kaufmann (2011)
13. Reijnders, D., Dinh, T.T.A.: On exploiting transaction concurrency to speed up blockchains (2020), <https://arxiv.org/abs/2003.06128>
14. Sei Documentation: Sei parallelization engine: Multi-core blockchain execution. Online documentation (2026), <https://docs.sei.io/learn/parallelization-engine>, accessed: 2026-06-14

15. Shavit, N., Touitou, D.: Software transactional memory. *Distributed Computing* **10**(2), 99–116 (1997). <https://doi.org/10.1007/s004460050028>
16. Solana Foundation: Sealevel — parallel processing thousands of smart contracts. Online documentation/blog post (2019), <https://solana.com/news/sealevel---parallel-processing-thousands-of-smart-contracts>, accessed: 2026-06-14
17. Sui Documentation: Types of object ownership. Online documentation (2026), <https://docs.sui.io/develop/objects/object-ownership/>, accessed: 2026-06-14
18. Valiant, L.G.: A bridging model for parallel computation. *Communications of the ACM* **33**(8), 103–111 (1990). <https://doi.org/10.1145/79173.79181>
19. Williams, S., Waterman, A., Patterson, D.: Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM* **52**(4), 65–76 (2009). <https://doi.org/10.1145/1498765.1498785>